Task 1: Test results

For {“normal” : (0, -100)}

```
PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 100
Enemy health: 0

GAME OVER: Player wins.

Process finished with exit code 0
```

For {"last stand": (-100, -30)}

```
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 0
Enemy health: 70

GAME OVER: Enemy wins.

Process finished with exit code 0
```

For {"last stand": (-100, -100)}

```
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 0
Enemy health: 0

DRAW.

Process finished with exit code 0
```

For {"last stand": (-100, -110)}

```
PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 0
Enemy health: -10

DRAW.

Process finished with exit code 0
```

Task 2: Test results

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear search execution time: 0.494513 seconds
Binary search execution time: 0.000175 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear search execution time: 1.201080 seconds
Binary search execution time: 0.000170 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear search execution time: 0.163947 seconds
Binary search execution time: 0.000166 seconds

Process finished with exit code 0
```

When searching through a large sorted list for a specific value, binary search takes a very consistent amount of time to compute, unlike linear search which can vary wildly. This is because linear search checks the first value in the list, and if it is not the target value, it moves to the next one. Because of this, we can expect to run the code in the for loop anywhere from 1 to 1000000 times, which is why the execution time varies wildly. Binary search meanwhile takes a very consistently small amount of time to execute. This is because binary search initially checks if the target value is greater than or less than the middle value of the list, and then checks whichever side the target value happens to fall on, before splitting that in two again and so on. Therefore, the maximum number of times the function can be called is equal to the

number of times the number equal to the size of the list can be divided by 2, rounded down (i.e. $\log_2(1000000)$, which is 19 when rounded down). Therefore, for a sorted list of this size, binary search is almost always going to be more efficient than linear search. It is worth noting that binary search is not functional on an unsorted list, in which case linear search is preferred.

Task 3: Test results

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Bubble Sort Execution Time: 2.188128 seconds
Insertion Sort Execution Time: 0.890205 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Bubble Sort Execution Time: 2.192528 seconds
Insertion Sort Execution Time: 0.878062 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Bubble Sort Execution Time: 2.175494 seconds
Insertion Sort Execution Time: 0.938036 seconds

Process finished with exit code 0
```

Both functions have a consistent run time, and insertion sort is consistently faster on the same list than bubble sort. This is because, while both functions use a for loop with a nested loop, bubble sort uses more assignments in its nested loop than insertion sort. In bubble sort, we loop over a part of the list, and swap elements depending on whether the first element is larger than

the following element. As there are two elements swapping, they are each assigned to each other, meaning a total of two assignments. However, the while loop in insertion sort only assigns one element per iteration. For this reason, insertion sort is more efficient than bubble sort.

Task 4: Test results

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear Insertion Sort Execution Time: 0.882438 seconds
Linear Bubble Sort Execution Time: 2.169161 seconds
Binary Insertion Sort Execution Time: 0.876824 seconds
Binary Bubble Sort Execution Time: 2.157610 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear Insertion Sort Execution Time: 0.884848 seconds
Linear Bubble Sort Execution Time: 2.175996 seconds
Binary Insertion Sort Execution Time: 0.887722 seconds
Binary Bubble Sort Execution Time: 2.165327 seconds

Process finished with exit code 0
```

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Linear Insertion Sort Execution Time: 0.888884 seconds
Linear Bubble Sort Execution Time: 2.165828 seconds
Binary Insertion Sort Execution Time: 0.886752 seconds
Binary Bubble Sort Execution Time: 2.171331 seconds

Process finished with exit code 0
```

When we compare the results of the combined sort-searches, we get times that are comparable with the difference in times between the two sort algorithms. This is logical, as the search algorithms require a comparatively small amount of computation. When using these combined algorithms, it is preferable to be using the insertion sort function, just as it was before. We are

more interested in the result of sorting followed by linear or binary searching, as opposed to linear searching. These results are below:

```
/Users/alexr/PycharmProjects/SoftwareTech2/.venv/bin/python /Users/alexr/PycharmProjects
Binary Insertion Sort Execution Time: 0.885123 seconds
Binary Bubble Sort Execution Time: 2.172098 seconds
Linear Search Execution Time: 0.000212 seconds

Process finished with exit code 0
```

As can be seen, linear search is substantially faster than either combination of binary search and sorting methods, due to the complexity of both sorting algorithms. For this reason, unless a given list is already sorted, it is preferable for a search algorithm to be linear.